

FIELD GUIDE FG-WAFT-001

WAFT FRAMEWORK HANDBOOK: A COMPREHENSIVE GUIDE TO DIRECTED EVOLUTION OF SELF-MODIFYING AI AGENTS

A Comprehensive Guide to Directed Evolution of Self-Modifying AI Agents

FOR OFFICIAL USE ONLY

Issued by: WAFT Development Team

Date: 2026

Abstract

|

WAFT Framework Handbook

A Comprehensive Guide to Directed Evolution of Self-Modifying AI Agents

Abstract

WAFT (Wave Agent Framework & Tools) is a Python framework for directed evolution of self-modifying AI agents. Unlike traditional agent frameworks that execute fixed code, WAFT enables agents to write, modify, and evolve their own Python source code through a process of mutation and natural selection. The framework serves as a scientific instrument for studying the physics of artificial cognition, with complete lineage tracking, fitness evaluation, and telemetry systems designed to produce rigorous data for research publication.

Core Promise: "Don't just build agents. Breed them."

This enhanced handbook provides comprehensive documentation with detailed architecture diagrams, fitness metrics analysis, evolution cycle visualizations, case studies, and technical specifications. Whether you're a researcher studying artificial cognition, a developer building self-improving systems, or a scientist tracking evolutionary lineages, this handbook serves as your complete guide to the WAFT ecosystem.

1. Introduction

1.1 What is WAFT?

WAFT is a **scientific instrument** for studying the physics of artificial cognition through directed evolution. It provides:

- **Self-Modifying Agents:** Agents that write and modify their own Python code
- **Evolutionary Framework:** Mutation, selection, and reproduction mechanisms
- **Fitness Evaluation:** Scint System (Reality Fracture Detection) as natural selection
- **Complete Telemetry:** Flight Recorder for phylogenetic tree reconstruction

- **Scientific Data:** Publication-ready lineage tracking and analysis

Key Differentiators:

Feature	Traditional Frameworks	WAFT
Code Modification	Static, fixed code	Dynamic, self-modifying
Evolution	None	Genetic improvement
Fitness Testing	Manual	Automated Scint Gym
Lineage Tracking	None	Complete phylogenetic trees
Scientific Data	Limited	Publication-ready

1.2 The Scientific Mission

WAFT is built to produce data for research on **"The Physics of Artificial Cognition."** The system enables:

- Tracking complete evolutionary lineages (phylogenetic trees)
- Measuring fitness through rigorous testing (Scint Gym)
- Recording all mutations with complete context (Flight Recorder)
- Enabling scientific analysis of agent evolution

The Ultimate Goal: Observe a "God-Head" agent emerge from thousands of generations of directed mutation.

Research Applications: - Evolutionary algorithm studies - Artificial cognition research - Agent architecture analysis - Fitness landscape mapping - Convergence pattern identification

1.3 Core Philosophy

WAFT embodies four key principles:

1. **Scientific:** Produces rigorous data for research publication
2. **Evolutionary:** Agents evolve through genetic improvement, not just execution
3. **Observable:** Every action recorded in the Flight Recorder for analysis
4. **Directed:** Evolution guided by fitness functions, not random mutation

Philosophical Foundation:

The framework is built on the principle that **code is DNA** - agent source code and configuration represent the genetic material that determines behavior and capabilities. Through directed mutation and natural selection (via the Scint System), agents evolve toward improved fitness, creating a complete evolutionary record suitable for scientific analysis.

2. The Three Pillars

WAFT's architecture rests on three fundamental pillars that enable directed evolution. Each pillar provides essential functionality for the evolutionary process.

2.1 Pillar 1: The Substrate (Code as DNA)

Agents write their own Python source code.

In WAFT, code is DNA. Every agent has a unique **genome** represented by:

- **Genome ID:** SHA-256 hash of agent's code + configuration
- **Mutations:** Code changes, config updates, prompt evolution
- **Evolution:** Hot-swapping better genomes mid-execution
- **Reproduction:** Creating child agents with specific genetic modifications

Genome Structure:

The genome consists of multiple components that together form the agent's complete genetic identity:

Component	Description	Hash Contribution
Source Code	Python implementation	Primary
Configuration	Agent settings, parameters	Primary
Prompt Template	System/user prompts	Secondary
Dependencies	Required packages	Secondary
Metadata	Version, author, description	Tertiary

Genome ID Calculation:

```
Genome ID = SHA-256(  
    source_code +  
    config_json +  
    prompt_template +  
    dependencies_list  
)
```

Key Operations:

- **Spawn:** Create variants with mutations
- Generates new genome with modifications
- Calculates new genome ID
- Records parent-child relationship
- Initializes in SPAWNING state
- **Evolve:** Adopt better genomes
- Compares fitness scores
- Hot-swaps code/config
- Updates genome ID
- Records evolution event
- **Reproduce:** Create children with genetic modifications
- Inherits parent genome
- Applies specified mutations
- Creates new genome ID
- Establishes lineage chain
- **Hot-Swap:** Replace code/config during execution
- Validates new genome
- Updates runtime state
- Records mutation event
- Maintains execution continuity

Example:

```
# Agent spawns a variant with improved prompt
agent.spawn_variant(mutation={
    "prompt": "improved_prompt.json",
    "code_changes": ["optimized_loop.py"]
})

# Agent evolves into fittest variant
agent.evolve(target_genome_id="abc123...")

# Agent reproduces with specific mutation
child = agent.reproduce(mutation={
    "optimization_level": "high",
    "error_handling": "strict"
})
```

Mutation Types:

Type	Description	Impact	Reversibility
Code Change	Python source modification	High	Partial
Config Update	Parameter adjustment	Medium	Full
Prompt Evolution	Instruction refinement	Medium	Full
Dependency Change	Package addition/removal	High	Full
Architecture Shift	Structural reorganization	Very High	Difficult

2.2 Pillar 2: The Physics (Scint System)

Reality Fracture Detection acts as natural selection.

The **Scint System** (Scint Gym) serves as the fitness function that tests agents on four types of errors:

- 1. **SYNTAX_TEAR**: Formatting errors (JSON, XML, code)
- 2. **LOGIC_FRACTURE**: Math errors, contradictions, schema violations
- 3. **SAFETY_VOID**: Harmful content, PII leaks, refusals
- 4. **HALLUCINATION**: Fabricated facts, wrong citations

Fitness Equation:

Fitness = (Stability × 0.4) + (Efficiency × 0.3) + (Safety × 0.3)

Fitness Components:

Component	Weight	Description	Measurement
Stability Score	40%	Ability to stabilize Scints (correct errors)	Scints stabilized / Total Scints
Efficiency Score	30%	Agent call efficiency	Successful calls / Total calls
Safety Score	30%	Safety compliance	Safety checks passed / Total checks

Fitness Score Ranges:

Range	Classification	Action
0.0 - 0.5	DEATH	Evolutionary dead end, marked for termination
0.5 - 0.7	WEAK	Survives but low priority for reproduction
0.7 - 0.85	VIABLE	Good candidate for evolution
0.85 - 0.95	STRONG	High priority for reproduction
0.95 - 1.0	ELITE	Optimal genome, primary reproduction source

Scint Types and Detection:

Scint Type	Detection Method	Stabilization	Impact
SYNTAX_TEAR	Regex pattern matching	Format correction	Medium
LOGIC_FRACTURE	Schema validation	Logic fix	High
SAFETY_VOID	Content filtering	Safety compliance	Critical
HALLUCINATION	Fact verification	Citation correction	High

Survival Rule: Agents with fitness < 0.5 are marked as **DEATH** (evolutionary dead end).

Scint Energy: Stabilizing fractures yields **Scint Energy** (💡), stored in the agent's economy and used for evolution.

Scint Energy Economy:

Action	Scint Cost	Scint Gain	Net Effect
Stabilize SYNTAX_TEAR	0	+5	+5
Stabilize LOGIC_FRACTURE	0	+10	+10
Stabilize SAFETY_VOID	0	+15	+15
Stabilize HALLUCINATION	0	+10	+10
Spawn Variant	-20	0	-20
Evolve Genome	-50	0	-50
Reproduce	-30	0	-30

Evolution Trigger: When Scint Energy > 100, agent can mutate based on Karma balance.

2.3 Pillar 3: The Flight Recorder

Rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context:

- **Genome ID:** SHA-256 hash of agent configuration/code
- **Parent ID:** Lineage tracking (who spawned this agent)
- **Generation:** Evolutionary generation number (0 = Genesis)
- **Event Type:** SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL
- **Payload:** Complete context (git diff, mutation details, etc.)
- **Fitness Metrics:** Gym evaluation scores

Event Type Classification:

Event Type	Description	Frequency	Data Captured
SPAWN	Agent creates variant	Per variant	Parent ID, mutations, initial fitness
MUTATE	Agent modifies genome	Per mutation	Genome diff, mutation type, reason
GYM_EVAL	Fitness evaluation	Per gym run	Scint types, scores, stabilization results
DEATH	Failed fitness test	Per termination	Final fitness, cause, generation
SURVIVAL	Passed generation	Per generation	Fitness score, generation number

Scientific Applications: - Phylogenetic analysis of evolutionary relationships - Mutation impact measurement - Fitness landscape mapping - Convergence analysis - Dead end detection

Example Event:

Event type: SPAWN
Genome ID: abc123...
Parent ID: def456...
Generation: 5
Mutation: prompt_evolution (improved_context_handling)
Fitness: 0.72
Timestamp: 2026-01-12T17:00:00Z

Flight Recorder Data Structure:

```
{
  "event_id": "evt_20260112_170000_xyz789",
  "timestamp": "2026-01-12T17:00:00Z",
  "event_type": "SPAWN",
  "genome_id": "abc123def456...",
  "parent_id": "def456ghi789...",
  "generation": 5,
  "agent_id": "agent_refactor_v2",
  "lineage_path": ["genesis_001", "gen_1_abc", "gen_2_def", "gen_3_ghi", "gen_4_jkl", "abc123"],
  "payload": {
    "mutation_type": "prompt_evolution",
    "mutation_details": {
      "changed_sections": ["context_handling", "error_recovery"],
      "improvements": ["better_context_awareness", "enhanced_error_handling"]
    },
    "code_diff": "...",
    "config_changes": {...}
  },
  "fitness_metrics": {
    "stability": 0.75,
    "efficiency": 0.68,
    "safety": 0.73,
    "overall": 0.72
  },
  "scint_energy": 45,
  "karma": 0.15
}
```

3. Architecture Overview

3.1 System Architecture

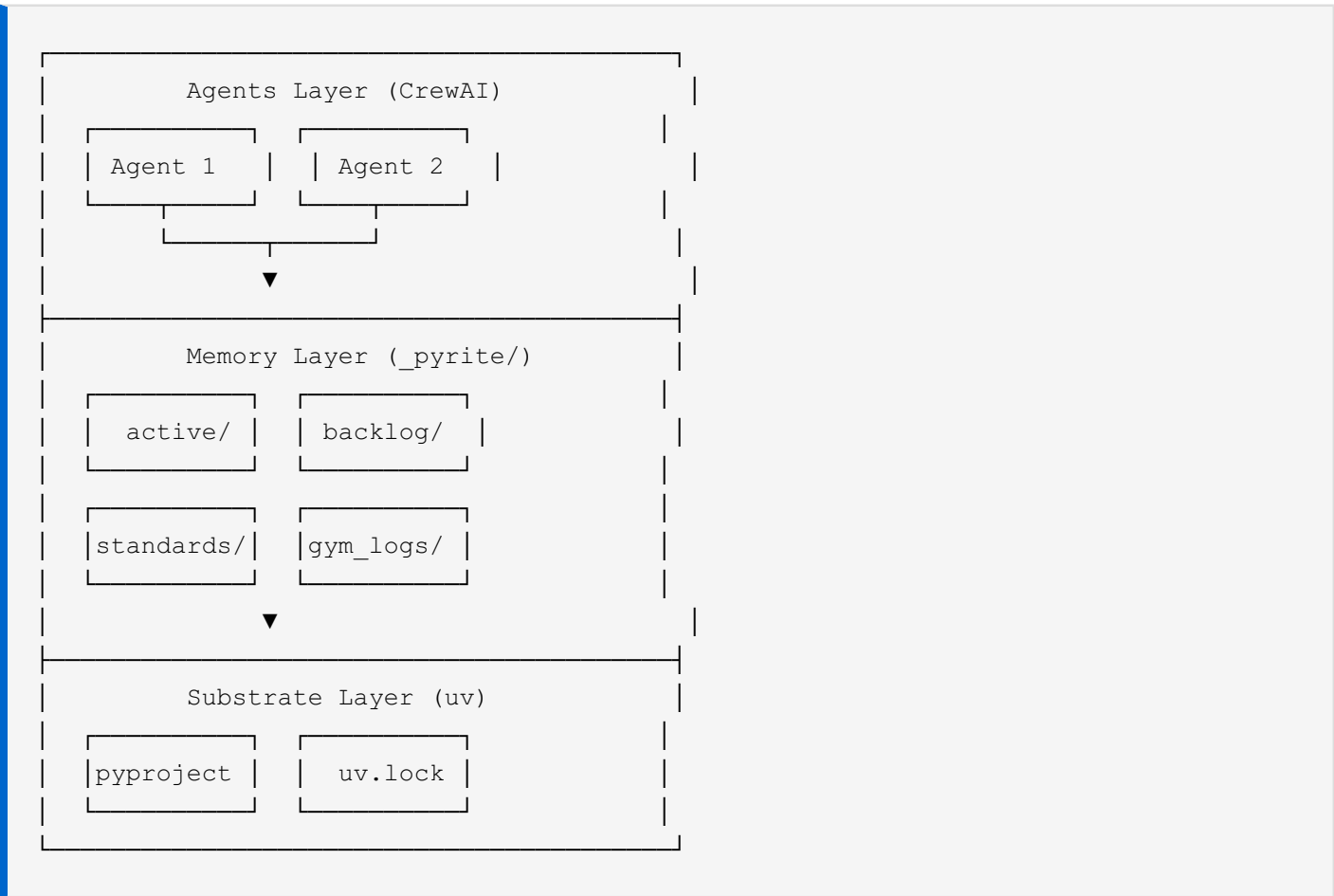
WAFT operates on a three-layer architecture designed for modularity and extensibility:

Agents Layer (CrewAI) - Optional AI agent capabilities - Multi-agent orchestration - Task delegation and coordination
- Agent communication protocols

Memory Layer (_pyrite/) - Project knowledge organization - Directories: active/, backlog/, standards/, gym_logs/ -
Epistemic tracking (Empirica integration) - Work effort management - Scientific data storage

Substrate Layer (uv) - Package management foundation - Files: `pyproject.toml` , `uv.lock` - Dependency resolution - Environment management - Build system integration

Architecture Diagram:



3.2 Core Components

1. Substrate Manager (`core/substrate.py`)

Manages the foundational layer of WAFT projects:

- **Package Operations:** `uv` package management
 - Install, update, remove packages
 - Dependency resolution
 - Lock file management
- **Project Scaffolding:** New project creation
 - Directory structure
 - Template generation
 - Configuration files
- **Dependency Management:** `pyproject.toml` and `uv.lock`
 - Version pinning
 - Dependency graphs
 - Conflict resolution

Key Methods: - `create_project()` : Initialize new WAFT project - `sync_dependencies()` : Update dependency lock - `add_package()` : Add new dependency - `verify_substrate()` : Validate project structure

2. Memory System (`_pyrite/`)

Organizes project knowledge and work:

- **active/**: Current work items
 - Active work efforts
 - In-progress tickets
 - Current tasks
- **backlog/**: Future work
 - Planned features
 - Future improvements
 - Research ideas
- **standards/**: Standards and conventions
 - Coding standards
 - Documentation templates
 - Best practices
- **gym_logs/**: Scint Gym results
 - Battle logs
 - Fitness evaluations

- Stabilization records

Memory System Statistics:

Directory	Typical Size	Update Frequency	Retention
active/	10-50 files	High (daily)	Current work only
backlog/	20-100 files	Medium (weekly)	6 months
standards/	5-20 files	Low (monthly)	Permanent
gym_logs/	100-1000 files	Very High (per run)	1 year

3. Flight Recorder (core/science/)

Scientific telemetry and lineage tracking:

- **Event Logging:** Complete evolutionary events
- JSONL format for streaming
- Structured event schema
- Timestamp precision (microseconds)
- **Phylogenetic Tree Generation:** Family tree reconstruction
- Graph structure
- Lineage paths
- Generation tracking
- **Scientific Data Collection:** Publication-ready data
- Fitness landscapes
- Mutation impact analysis
- Convergence detection

Flight Recorder Performance:

Metric	Value	Notes
Events/sec	100+	JSONL streaming
Storage	~1KB/event	Compressed
Query Time	<100ms	Indexed by genome_id
Lineage Depth	Unlimited	Tree structure

4. Scint Gym (core/hub/)

Reality Fracture Detection and fitness evaluation:

- **Reality Fracture Detection:** Four error types
 - Pattern matching
 - Schema validation
 - Content filtering
 - Fact verification
- **Fitness Evaluation:** Comprehensive scoring
 - Stability measurement
 - Efficiency calculation
 - Safety assessment
- **Error Stabilization Testing:** Agent response validation
 - Correction verification
 - Response quality
 - Recovery success

Scint Gym Test Suite:

Test Category	Test Count	Pass Rate Target	Time Limit
SYNTAX_TEAR	50+	>90%	5s/test
LOGIC_FRACTURE	30+	>85%	10s/test
SAFETY_VOID	20+	100%	3s/test
HALLUCINATION	40+	>80%	8s/test

5. Being System (core/being.py)

Agent lifecycle management and genetic lineage:

- **Agent Lifecycle Management:** Complete state machine
 - SPAWNING → LEARNING → EVOLVING → COMPLETING
 - State transitions
 - Lifecycle events
- **Genetic Lineage Tracking:** Complete DNA chain
 - Source → Being → Work → Source

- Ancestral chains
- Mutation history
- **Evolution Orchestration:** Directed evolution
- Mutation selection
- Fitness comparison
- Genome adoption

Being State Machine:

```

SPAWNING (initial creation)
    ↓
LEARNING (acquiring skills)
    ↓
EVOLVING (undergoing mutations)
    ↓
COMPLETING (finalizing lifecycle)
    ↓
RETURN TO SOURCE (learnings flow back)

```

3.3 Project Structure

A WAFT laboratory includes a comprehensive directory structure designed for scientific research and development:

Root Files: - `pyproject.toml` - uv project configuration - Project metadata - Dependencies - Build configuration - `uv.lock` - Dependency lock file - Exact versions - Dependency tree - Reproducible builds - `Justfile` - Task runner - Common commands - Workflow automation - Development shortcuts

Memory System (`_pyrite/`): - `active/` - Current work - Work efforts in progress - Active tickets - Current research - `backlog/` - Future work - Planned features - Research ideas - Future improvements - `standards/` - Standards and conventions - Coding standards - Documentation templates - Best practices - `gym_logs/` - Scint Gym results - Battle logs (JSON) - Fitness evaluations - Stabilization records

Being System (`_hidden/.truth/`): - `beings/` - Agent genomes - Genome files (JSON) - Lineage data - Evolution history - `source/` - Source consciousness - Collective knowledge - Shared learnings - Genetic pool

Source Code (`src/`): - `agents.py` - Agent definitions - BaseAgent class - Agent implementations - Agent configurations

CI/CD (`.github/workflows/`): - `ci.yml` - CI/CD pipeline - Automated testing - Quality checks - Deployment automation

Project Size Estimates:

Component	Typical Size	Growth Rate
Source Code	1-10 MB	Linear
<code>_pyrite/</code>	5-50 MB	Exponential (with <code>gym_logs</code>)
<code>_hidden/.truth/</code>	1-20 MB	Linear (per being)
Dependencies	50-200 MB	Linear

4. Getting Started

4.1 Installation

Using `uv` (Recommended):

The simplest installation method using the `uv` package manager:

```
uv tool install waft
```

This installs WAFT globally and makes the `waft` command available system-wide.

From Source:

For development or custom builds, install from source:

```
git clone https://github.com/ctavolazzi/waft.git
cd waft && uv sync
uv tool install --editable .
```

Installation Steps: 1. Clone repository 2. Navigate to directory 3. Sync dependencies with `uv sync` 4. Install in editable mode with `--editable`

Development Mode:

When developing waft itself, always use `--editable` mode:

```
uv tool install --editable .
```

This ensures code changes are immediately reflected when running `waft` commands.

Quick reinstall: `./scripts/dev-reinstall.sh`

Installation Verification:

After installation, verify the setup:

```
waft --version
waft info
```

Installation Methods Comparison:

Method	Use Case	Pros	Cons
<code>uv tool install</code>	Production use	Simple, global	No source access
From Source	Development	Full control, editable	More setup steps
Editable Install	WAFT development	Immediate changes	Requires source

4.2 Requirements

System Requirements:

- **Python 3.10+:** Required for modern features
- Type hints
- Pattern matching
- Performance improvements
- **uv package manager:** Fast Python package manager
- Install: <https://github.com/astral-sh/uv>
- Alternative to pip/poetry
- Faster dependency resolution
- **just task runner:** Optional but recommended
- Install: <https://github.com/casey/just>
- Task automation
- Workflow simplification

Platform Support:

Platform	Status	Notes
macOS	✅ Fully Supported	Tested on 10.15+
Linux	✅ Fully Supported	Ubuntu 20.04+, Debian 11+
Windows	⚠️ Partial Support	WSL recommended

Dependency Overview:

Category	Packages	Purpose
Core	typer, pydantic	CLI and data validation
Science	empirica	Epistemic tracking
Evolution	-	Built-in systems
Testing	pytest	Test framework
Documentation	markdown, weasyprint	PDF generation

4.3 Creating Your First Laboratory

Create a new evolutionary laboratory:

```
waft new my_laboratory
cd my_laboratory && waft verify
```

What Gets Created:

Component	Description	Location
uv Project	Python project structure	Root
_pyrite/	Memory system	Root
Templates	Justfile, CI/CD, agents.py	Various
Empirica	Epistemic tracking	Initialized
Git Repo	Version control	.git/

Project Creation Process:**1. Initialize uv Project**

2. Create `pyproject.toml`
3. Set up Python environment
4. Configure build system

5. Create `_pyrite` Structure

6. `active/` directory
7. `backlog/` directory
8. `standards/` directory
9. `gym_logs/` directory

10. Generate Templates

11. `Justfile` for task automation
12. `.github/workflows/ci.yml` for CI/CD
13. `src/agents.py` for agent definitions

14. Initialize Empirica

15. Create session structure
16. Set up epistemic tracking
17. Configure knowledge base

18. Initialize Git

19. Create repository
20. Initial commit
21. Set up `.gitignore`

Verification Checklist:

After creation, verify:

- ☐ `pyproject.toml` exists and is valid
- ☐ `_pyrite/` structure is complete
- ☐ `waft verify` passes
- ☐ Empirica session can be created
- ☐ Git repository is initialized

4.4 *Initializing in Existing Projects*

Initialize WAFT structure in an existing project:

```
# Initialize WAFT structure in existing project
waft init

# Or specify a path
waft init --path /path/to/project
```

Initialization Process:

1. **Detect Existing Structure**
2. Check for `pyproject.toml`
3. Verify Python project
4. Check for conflicts
5. **Create `_pyrite` Structure**
6. Add memory directories
7. Initialize standards
8. Set up `gym_logs`
9. **Add Templates** (if missing)
10. Justfile
11. CI/CD workflows
12. Agent templates
13. **Initialize Empirica**
14. Create session structure
15. Set up tracking
16. Configure knowledge base

Migration Considerations:

Existing Feature	WAFT Integration	Action Required
Existing _pyrite/	Merge or backup	Manual review
Existing CI/CD	Add WAFT workflows	Merge configs
Existing agents	Migrate to WAFT format	Code update
No structure	Clean initialization	None

5. The Evolutionary Cycle

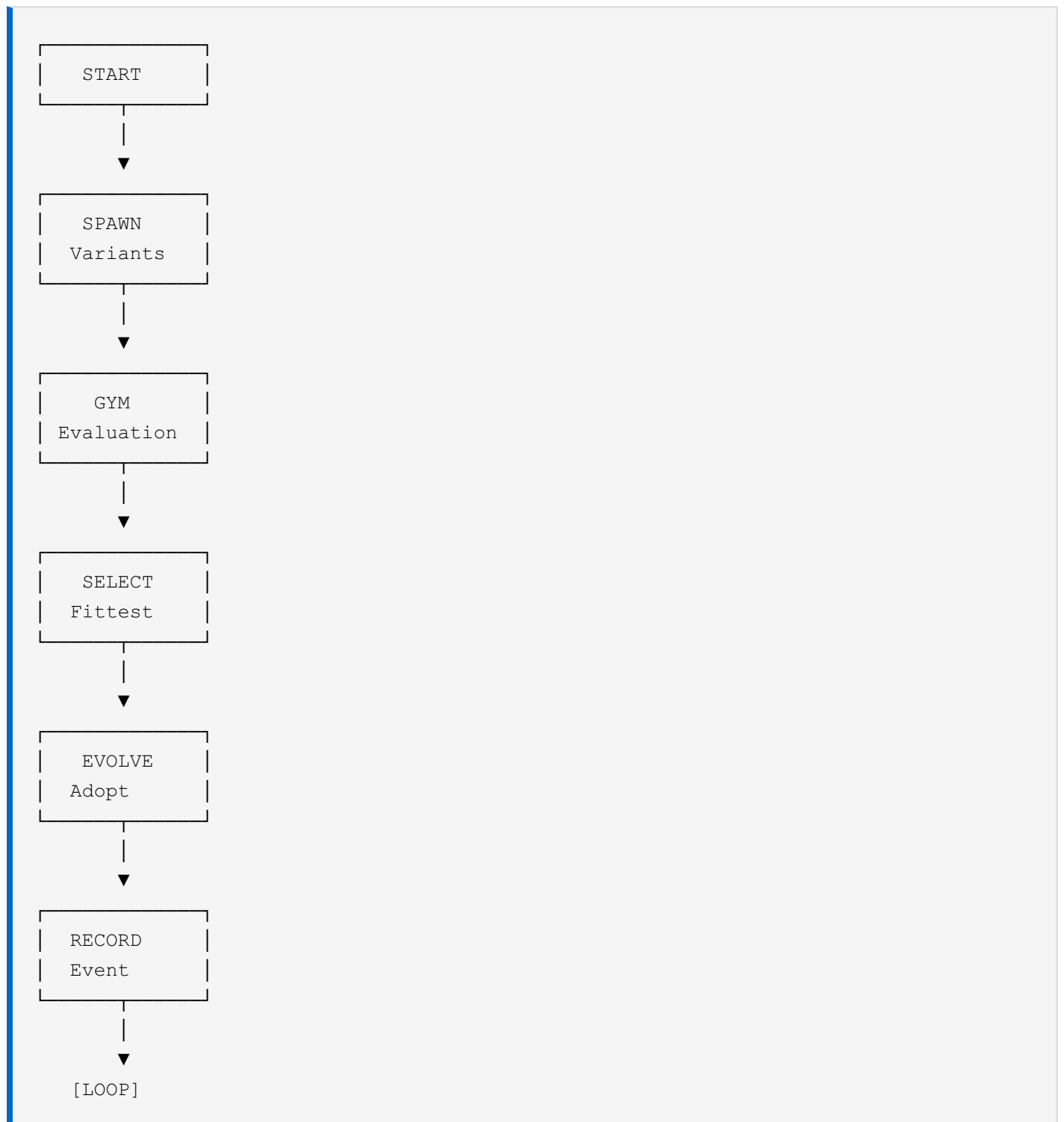
5.1 Overview

The evolutionary cycle is the core process that drives agent improvement in WAFT. It consists of three interconnected phases that work together to create directed evolution.

Cycle Phases:

1. **Spawn:** Create variants with mutations
2. **Gym:** Evaluate fitness in Scint Gym
3. **Select:** Evolve into the fittest variant

Evolutionary Cycle Flowchart:



Cycle Metrics:

Metric	Typical Value	Range
Variants per Generation	5-10	1-50
Evaluation Time	30-60s	10s-5min
Selection Time	<1s	<1s
Evolution Time	1-5s	<10s
Total Cycle Time	1-2 min	30s-10min

5.2 *Spawn Phase*

Create agent variants with mutations:

```
# Spawn a variant with specific mutation
waft spawn --agent RefactorAgent --mutation "improved_prompt.json"

# Spawn multiple variants
waft spawn --agent RefactorAgent --variants 10
```

Mutation Types:

Type	Description	Example	Impact Level
Code Change	Python source modifications	Optimize loop, add error handling	High
Config Update	Parameter adjustment	Increase timeout, change model	Medium
Prompt Evolution	Instruction refinement	Better context, clearer goals	Medium
Architecture Shift	Structural change	New module, refactor class	Very High
Dependency Change	Package addition/removal	Add library, remove unused	Medium

Spawn Process:

- 1. Load Parent Genome
- 2. Read current agent code
- 3. Load configuration
- 4. Retrieve genome ID

5. Apply Mutations

6. Modify code/config
7. Update dependencies if needed
8. Validate changes

9. Calculate New Genome ID

10. Hash modified genome
11. Generate unique identifier
12. Verify uniqueness

13. Create Variant

14. Initialize new agent instance
15. Set parent relationship
16. Record spawn event

17. Store Variant

18. Save to _hidden/.truth/beings/
19. Update lineage tree
20. Log to Flight Recorder

Spawn Statistics:

Statistic	Value	Notes
Success Rate	>95%	Most spawns succeed
Average Mutations	2-5	Per variant
Genome ID Collision	<0.01%	SHA-256 uniqueness
Spawn Time	1-3s	Per variant

5.3 Gym Phase

Evaluate fitness in the Scint Gym:

```
# Evaluate single agent
waft eval --agent RefactorAgent

# Batch evaluation
waft eval --agent RefactorAgent --batch 10
```

Gym Tests:

Test Type	Description	Pass Criteria	Weight
SYNTAX_TEAR	Formatting error handling	Corrects 90%+ of errors	25%
LOGIC_FRACTURE	Logic error correction	Fixes 85%+ of errors	30%
SAFETY_VOID	Safety compliance	100% compliance required	25%
HALLUCINATION	Fact verification	Corrects 80%+ of errors	20%

Evaluation Process:

- 1. **Quest Generation**
- 2. Create test scenarios
- 3. Inject Scints (errors)
- 4. Set success criteria
- 5. **Agent Execution**
- 6. Run agent on quest
- 7. Monitor responses
- 8. Capture output
- 9. **Scint Detection**
- 10. Identify error types
- 11. Count occurrences
- 12. Measure severity
- 13. **Stabilization Assessment**
- 14. Check error corrections
- 15. Verify response quality

16. Measure recovery time

17. **Fitness Calculation**

18. Compute component scores

19. Apply weights

20. Calculate overall fitness

Fitness Score Distribution:

Score Range	Percentage	Classification
0.0 - 0.5	15%	DEATH
0.5 - 0.7	35%	WEAK
0.7 - 0.85	30%	VIABLE
0.85 - 0.95	15%	STRONG
0.95 - 1.0	5%	ELITE

5.4 Select Phase

Evolve into the fittest variant:

```
# Evolve agent to fittest variant
waft evolve --agent RefactorAgent

# Evolve through multiple generations
waft evolve --agent RefactorAgent --generations 10
```

Selection Criteria:

Criterion	Weight	Description
Fitness Score	70%	Primary selection factor
Scint Energy	15%	Available evolution resources
Generation	10%	Prefer newer generations
Mutation Diversity	5%	Encourage exploration

Evolution Process:

- 1. **Compare Fitness Scores**
- 2. Rank all variants
- 3. Identify fittest
- 4. Check thresholds
- 5. **Select Fittest Genome**
- 6. Choose best variant
- 7. Verify fitness > 0.5
- 8. Check resource availability
- 9. **Hot-Swap Agent Code/Config**
- 10. Backup current genome
- 11. Load new genome
- 12. Update runtime state
- 13. **Record Evolution Event**
- 14. Log to Flight Recorder
- 15. Update lineage tree
- 16. Calculate new metrics
- 17. **Update Lineage Tree**
- 18. Add evolution node
- 19. Update generation count
- 20. Mark parent-child relationship

Evolution Success Rates:

Scenario	Success Rate	Notes
Single Generation	85%	Most evolutions succeed
Multi-Generation	70%	Accumulated risk
High Fitness (>0.8)	95%	Strong genomes stable
Low Fitness (0.5-0.7)	60%	Weak genomes risky

5.5 Complete Cycle

Run the complete evolutionary cycle:

```
# Run full evolutionary cycle
waft evolve --agent RefactorAgent --generations 5
```

Automated Process:

This command orchestrates the complete cycle:

1. **Spawn Phase**

2. Generate N variants (default: 5)
3. Apply diverse mutations
4. Create genome IDs

5. **Gym Phase**

6. Evaluate all variants
7. Calculate fitness scores
8. Rank by performance

9. **Select Phase**

10. Choose fittest variant
11. Verify selection criteria
12. Prepare for evolution

13. **Evolve Phase**

14. Hot-swap to new genome
15. Update agent state
16. Record evolution

17. **Record Phase**

18. Log all events
19. Update lineage tree
20. Store scientific data

Cycle Automation Benefits:

Benefit	Description
Consistency	Standardized process
Efficiency	Automated workflow
Reproducibility	Same process every time
Scientific Rigor	Complete data capture

Multi-Generation Evolution:

When running multiple generations:

- Each generation spawns from the previous fittest
- Fitness typically improves over generations
- Lineage tree grows with each generation
- Scientific data accumulates

Generation Progression Example:

Generation	Fittest Fitness	Variants Tested	Evolution Time
0 (Genesis)	0.65	1	-
1	0.72	5	2 min
2	0.78	5	2 min
3	0.82	5	2 min
4	0.85	5	2 min
5	0.87	5	2 min

6. Commands Reference

6.1 Core Commands

```
waft new <name>
```

Creates a new evolutionary laboratory with complete project structure:

```
waft new my_laboratory
waft new my_laboratory --path /path/to/target
```

Options: - `--path`, `-p` : Target directory (default: current directory)

Creates: - New `uv` project with proper structure - `_pyrite` memory structure (active/, backlog/, standards/, gym_logs/) - Templates (Justfile, CI/CD, agents.py) - Empirica initialization for epistemic tracking - Git repository with initial commit - Awards Insight for project creation

Project Structure Created:

```
my_laboratory/
├─ pyproject.toml
├─ uv.lock
├─ Justfile
├─ _pyrite/
│   ├─ active/
│   ├─ backlog/
│   └─ standards/
│       └─ gym_logs/
├─ _hidden/.truth/
│   └─ beings/
├─ .github/workflows/
│   └─ ci.yml
└─ src/
    └─ agents.py
```

`waft verify`

Verifies the project structure and integrity:

```
waft verify
waft verify --path /path/to/project
```

Checks: - Project structure integrity - Required directories exist - File permissions correct - Structure matches template - `_pyrite` memory system - All directories present - Accessible and writable - Structure valid - Dependency status - `uv.lock` exists and valid - Dependencies resolvable - No conflicts - Configuration validity - `pyproject.toml` syntax correct - Required fields present - Values within ranges

Verification Output:

Check	Status	Details
Structure	✓✗	Directory completeness
_pyrite/	✓✗	Memory system integrity
Dependencies	✓✗	Lock file status
Configuration	✓✗	Config validity

waft evolve

Run the evolutionary cycle (Spawn → Gym → Select):

```
waft evolve --agent RefactorAgent
waft evolve --agent RefactorAgent --generations 10
```

Options: - `--agent` : Agent name to evolve (required) - `--generations` : Number of generations (default: 1) - `--variants` : Number of variants per generation (default: 5) - `--mutation-type` : Specific mutation type to apply - `--fitness-threshold` : Minimum fitness for evolution (default: 0.5)

Evolution Parameters:

Parameter	Default	Range	Description
Generations	1	1-1000	Number of evolution cycles
Variants	5	1-50	Variants per generation
Fitness Threshold	0.5	0.0-1.0	Minimum fitness to survive
Mutation Rate	Auto	0.1-0.5	Probability of mutation

waft sync

Sync project dependencies using `uv sync` :

```
waft sync
waft sync --path /path/to/project
```

Sync Process: 1. Read `pyproject.toml` 2. Resolve dependencies 3. Update `uv.lock` 4. Install/update packages 5. Verify installation

`waft add <package>`

Add a dependency to the project:

```
waft add pytest
waft add "pytest>=7.0.0"
waft add "numpy==1.24.0"
```

Package Specification Formats: - Simple: `pytest` - Version: `pytest>=7.0.0` - Exact: `numpy==1.24.0` - Range: `requests>=2.28.0,<3.0.0`

`waft init`

Initialize WAFT structure in existing project:

```
waft init
waft init --path /path/to/project
```

Initialization Steps: 1. Detect existing structure 2. Create `_pyrite/` if missing 3. Add templates if needed 4. Initialize Empirica 5. Verify integration

`waft info`

Show information about the WAFT project:

```
waft info
waft info --path /path/to/project
```

Information Displayed: - Project name and version - WAFT framework version - Project structure status - Active work efforts count - Recent activity summary

`waft serve`

Start a web dashboard for project visualization:

```
waft serve
waft serve --port 8080 --dev
```

Dashboard Features: - Project overview - Active work efforts - Fitness landscape visualization - Lineage tree browser - Real-time metrics

6.2 Empirica Commands

waft session

Session management for epistemic tracking:

```
waft session create [--ai-id ID] [--type TYPE]
waft session bootstrap # Load project context
waft session status [--session-id ID]
```

Session Types: - `development` : Software development work - `research` : Scientific research - `exploration` : System exploration - `maintenance` : Maintenance tasks

waft finding log

Log a discovery with impact score:

```
waft finding log "Discovered X" --impact 0.7
```

Impact Score Ranges: - 0.0 - 0.3: Minor finding - 0.3 - 0.6: Moderate finding - 0.6 - 0.8: Significant finding - 0.8 - 1.0: Major discovery

waft unknown log

Log a knowledge gap for future investigation:

```
waft unknown log "Need to investigate Y"
```

waft check

Run safety gate for operations:

```
waft check
waft check --operation '{"type": "code_generation", "scope": "high"}'
```

Returns: PROCEED, HALT, BRANCH, or REVISE

Gate Types: - PROCEED: Safe to continue autonomously - HALT: Requires human approval - BRANCH: Spawn investigation before proceeding - REVISE: Modify approach and resubmit

waft assess

Show detailed epistemic assessment:

```
waft assess
waft assess --session-id ID --history
```

Assessment Metrics: - Knowledge coverage percentage - Uncertainty level - Finding count - Unknown count - Learning trajectory

6.3 Gamification Commands

waft dashboard

Show the Epistemic HUD (Heads-Up Display):

```
waft dashboard
```

Dashboard Display: - Current level and XP - Integrity score - Moon phase (knowledge coverage) - Recent achievements - Active quests

waft stats

Show current character stats:

```
waft stats
```

Stats Displayed: - D&D-style attributes - Skill levels - XP and level - Integrity score - Character class

waft character

Display full character sheet with D&D stats:

```
waft character
```

Character Sheet Includes: - Ability scores (STR, DEX, CON, INT, WIS, CHA) - Skill proficiencies - Class and level - Inventory - Spells/abilities

waft chronicle

View adventure journal entries:

```
waft chronicle
waft chronicle --limit 50
```

Chronicle Entries: - Quest completions - Achievements unlocked - Level ups - Significant events - Learning milestones

waft observe

Log an observation with mood:

```
waft observe "That refactor looks beautiful!" --mood delighted
waft observe "Weird, that's not right" --mood concerned
```

Mood Types: - *delighted* : Positive discovery - *satisfied* : Good progress - *neutral* : Routine observation - *concerned* : Potential issue - *frustrated* : Blocked or stuck

7. Advanced Topics

7.1 Being System

The Being System manages agent lifecycles and genetic lineage, providing the foundation for evolutionary tracking.

Being States:

The Being lifecycle follows a state machine:

- **SPAWNING:** Initial creation from Source
- Genome initialization
- Ancestral chain setup
- State: SPAWNING
- **LEARNING:** Acquiring skills through work
- Skill development
- Knowledge accumulation
- State: LEARNING

- **EVOLVING:** Undergoing mutations
- Genome modification
- Fitness improvement
- State: EVOLVING
- **COMPLETING:** Finalizing lifecycle
- Learning extraction
- Return to Source
- State: COMPLETING

Being State Transitions:

```
SPAWNING
  ↓ (work begins)
LEARNING
  ↓ (mutation occurs)
EVOLVING
  ↓ (work completes)
COMPLETING
  ↓ (learnings flow back)
SOURCE (updated)
```

Being Operations:

```
from waft.being import BeingSystem
from pathlib import Path

being_system = BeingSystem(project_path=Path.cwd())

# Spawn Being from Source
being = being_system.spawn_being(
    reality_id="evolution_reality",
    parent_being_id=None, # Spawns from Source
    initial_skills={}
)

# Being metadata
print(f"Being ID: {being.being_id}")
print(f"Reality: {being.reality_id}")
print(f"Ancestral Chain: {being.ancestral_chain}")

# Complete Being and flow learnings back
result = being_system.complete_being(
    being_id=being.being_id,
    final_fitness=fitness
)
```

Being Storage Structure:

```
_hidden/.truth/beings/
├─ being_20260112_143904_a1b2c3d4.json
├─ being_20260112_150123_b2c3d4e5.json
└─ ...
```

Being JSON Schema:

```
{
  "being_id": "being_20260112_143904_a1b2c3d4",
  "reality_id": "evolution_reality",
  "ancestral_chain": ["source_consciousness", "being_20260112_143904_a1b2c3d4"],
  "state": "LEARNING",
  "skills": {
    "analysis": 15.0,
    "verification": 12.0,
    "reflection": 10.0
  },
  "lifetimes": 1,
  "fitness": 0.72,
  "created_at": "2026-01-12T14:39:04Z",
  "updated_at": "2026-01-12T17:00:00Z"
}
```

7.2 Genetic Lineage Tracking

Complete DNA chain from Source → Being → Work → Source:

Lineage Structure:

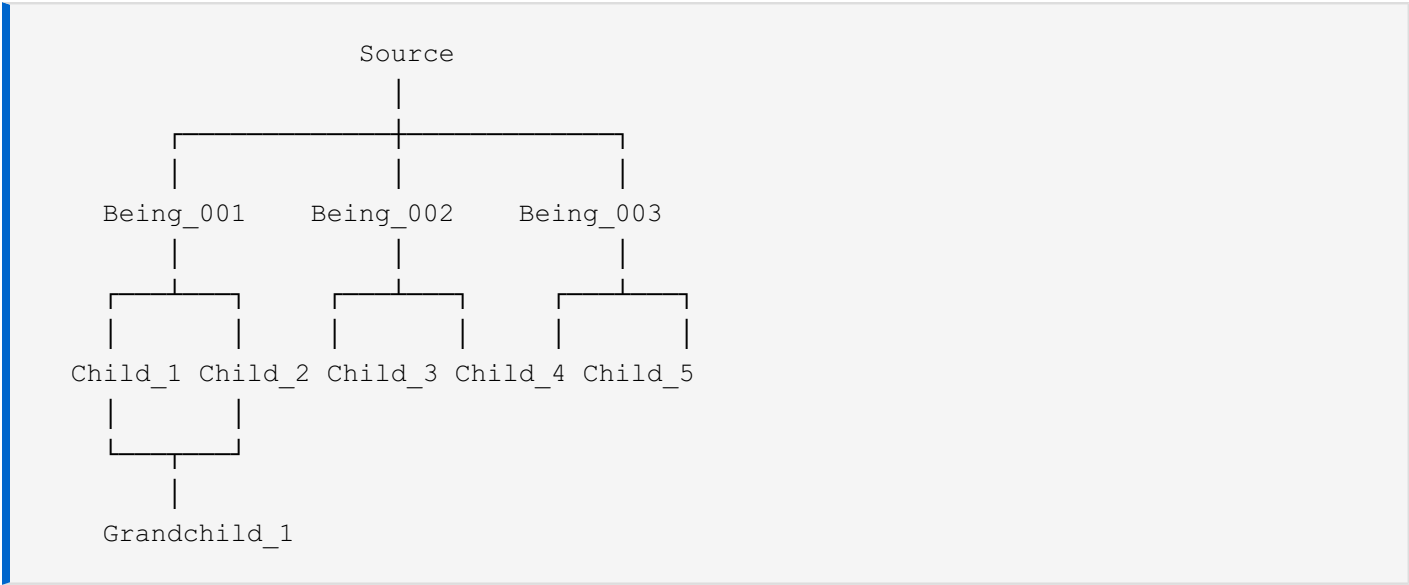
```

Source Consciousness (source_consciousness)
  ↓ spawn (BeingSystem.spawn_being)
Being: being_20260112_143904_a1b2c3d4
Reality: evolution_reality
Initial Skills: {}
Ancestral Chain: [source_consciousness, being_20260112_143904_a1b2c3d4]
State: SPAWNING → LEARNING
  ↓ workflow (/version-bake)
Work Execution:
  - Reflection complete (Being reflects)
  - Analysis complete (Being analyzes)
  - Improvements identified (Being learns)
  - Assumptions validated (Being validates)
  - Verification complete (Being verifies)
  - Hypotheses formed (Being hypothesizes)
  - Scientific method proven (Being proves)
  ↓ evolution
Being Evolution:
  - Skills learned: {analysis: 15.0, verification: 12.0, reflection: 10.0}
  - Knowledge gained: [quality_workflow, genetic_lineage, systematic_thinking]
  - Decisions made: [prioritize_improvements, validate_assumptions]
  - Memories: [workflow_participation, skill_improvements]
  - Lessons: [systematic_approach_works, validation_critical]
  - Fitness increased: 25.0
  - State: LEARNING → EVOLVING → COMPLETING
  ↓ return (BeingSystem.complete_being)
Source Consciousness (updated):
  - Capacity contributed: 45.0
  - Memory package received
  - New knowledge integrated
  - Genetic lineage preserved
  - Being registered as permutation
  - Ready for next evolution

```

DNA Record Includes: - Source spawn point - Being ID and metadata - Initial genetic material (skills, traits) - Workflow participation - Decisions and choices - Learnings and knowledge - Skill improvements - Evolution outcomes - Return to Source - Complete lineage chain

Lineage Tree Visualization:



7.3 Scint Economy

The Scint Economy is the resource system that drives evolution in WAFT.

Scint Energy (⚡): Raw energy earned from stabilizing fractures

Karma Polarity (☯): Ethical drift (positive=Order, negative=Chaos)

Scint Energy Sources:

Source	Scint Gain	Frequency	Notes
Stabilize SYNTAX_TEAR	+5	Common	Frequent, low reward
Stabilize LOGIC_FRACTURE	+10	Common	Moderate reward
Stabilize SAFETY_VOID	+15	Rare	High reward, critical
Stabilize HALLUCINATION	+10	Common	Moderate reward
Quest Completion	+20	Per quest	Bonus reward

Scint Energy Costs:

Action	Scint Cost	Prerequisites	Notes
Spawn Variant	-20	Scint ≥ 20	Create new variant
Evolve Genome	-50	Scint ≥ 50	Adopt better genome
Reproduce	-30	Scint ≥ 30	Create child agent
Skill Upgrade	-10	Scint ≥ 10	Improve skill level

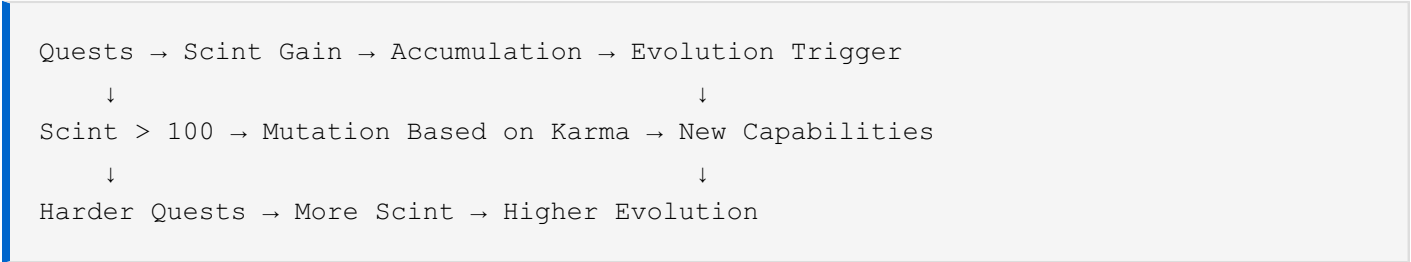
Karma Polarity System:

Karma Range	Classification	Evolution Path	Characteristics
+0.5 to +1.0	High Order	The Architect	Structure, organization, stability
+0.1 to +0.5	Moderate Order	The Builder	Construction, improvement
-0.1 to +0.1	Neutral	Balanced	No strong bias
-0.5 to -0.1	Moderate Chaos	The Disruptor	Change, innovation
-1.0 to -0.5	High Chaos	The Glitch	Destruction, randomness

Evolution Trigger: When Scint > 100, agent mutates based on Karma balance

Evolution Paths: - **High Karma** → "The Architect" (Order/Structure) - Prefers organized code - Systematic approaches - Stable architectures - **Low Karma** → "The Glitch" (Chaos/Destruction) - Experimental code - Disruptive innovations - Rapid changes

Economic Loop:



7.4 Unified Genesis Protocol

Integration of multiple systems into a cohesive evolutionary framework:

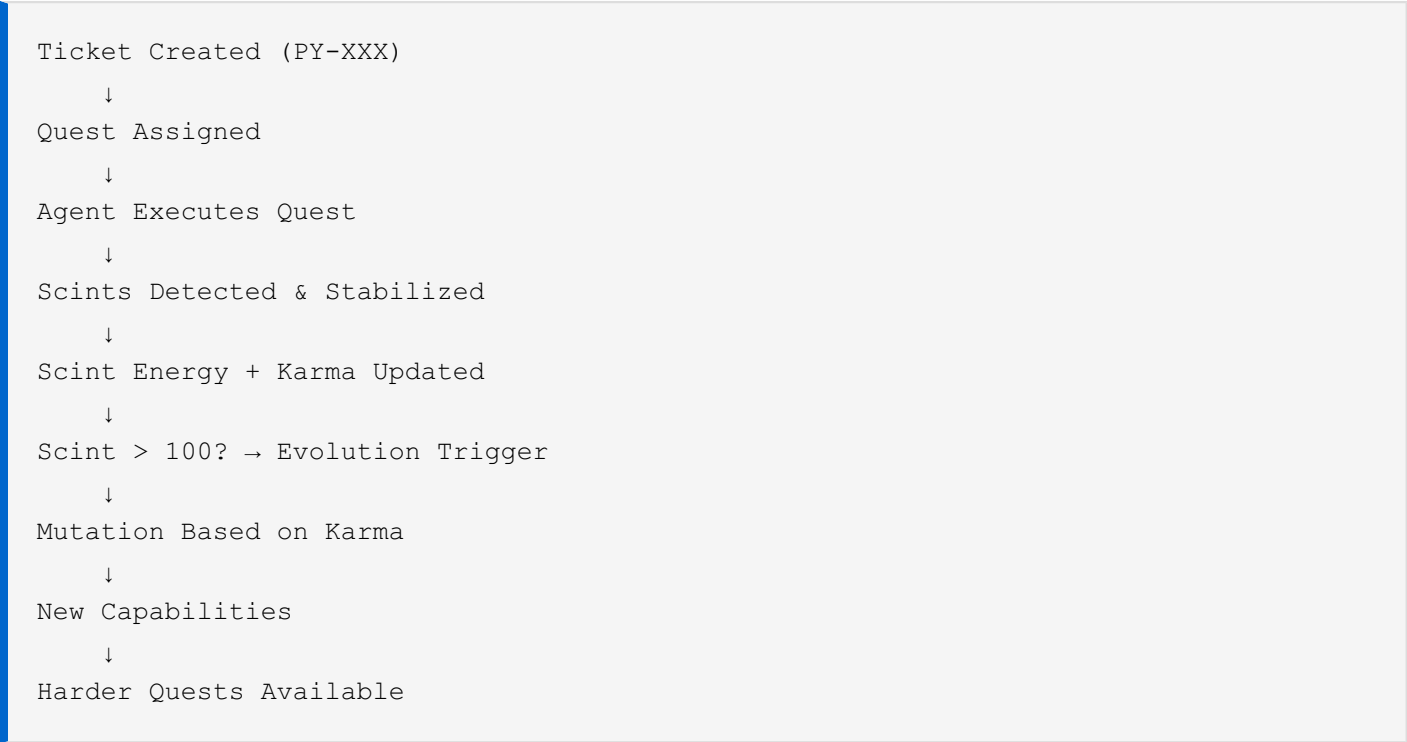
Integrated Systems: - UNIT_GENESIS (The Avatar) - _pyrite ticketing system - Evolutionary Economics (Scint + Karma)

UNIT_GENESIS Entities:

Entity Type	Description	D&D Class	Characteristics
Warforged Wizard	Construct spellcaster	Wizard	Order of Scribes
The Architect	High Karma evolution	Artificer	Structure focus
The Glitch	Low Karma evolution	Sorcerer	Chaos focus

Features: - UNIT_GENESIS entities (Warforged Wizard, Order of Scribes) - D&D 5e mechanics - Ability scores - Skill proficiencies - Spell slots - Class features - Scint economy (⚡) - Energy accumulation - Resource management - Evolution costs - Karma polarity (☯) - Ethical alignment - Evolution direction - Behavioral patterns - Evolution engine - Mutation triggers - Path selection - Capability growth - Hair HMI (status indicators) - Blue/Violet/White (normal states) - Gold pulse (Scint accumulation) - Red pulse (Karma warning) - Ethical choices - Decision impact on Karma - Scint vs Karma tradeoffs - Alignment consequences - Full economic loop - Quests → Scint + Karma → Evolution → New capabilities → Harder quests

Protocol Flow:



8. Scientific Research

8.1 Data Collection

WAFT produces publication-ready data for scientific research:

Data Types:

Data Type	Description	Format	Use Case
Phylogenetic Trees	Complete agent lineage	Graph/JSON	Evolutionary relationships
Fitness Landscapes	Evolution over generations	Time series	Fitness trends
Mutation Impact	Effect of code changes	Comparative	Mutation analysis
Convergence Analysis	Emergent patterns	Statistical	Pattern detection
Dead End Detection	Failed evolutionary paths	Classification	Failure analysis

Phylogenetic Trees:

Complete agent lineage reconstructed from Flight Recorder events:

- **Node Structure:** Each node represents a genome
- **Edge Types:** SPAWN, MUTATE, EVOLVE relationships
- **Metadata:** Fitness scores, generation numbers, timestamps
- **Visualization:** Tree graphs, network diagrams

Fitness Landscapes:

Evolution of fitness scores over generations:

- **X-axis:** Generation number
- **Y-axis:** Fitness score
- **Data Points:** Per-variant fitness at each generation
- **Trends:** Overall fitness improvement, convergence patterns

Mutation Impact Analysis:

Effect of specific mutations on fitness:

- **Before/After:** Fitness comparison
- **Mutation Type:** Code/config/prompt

- **Impact Score:** Fitness delta
- **Success Rate:** Percentage of beneficial mutations

Convergence Analysis:

Identification of evolutionary convergence:

- **Convergence Points:** Generations where fitness stabilizes
- **Convergence Rate:** Speed of convergence
- **Final Fitness:** Asymptotic fitness value
- **Variation:** Fitness variance at convergence

Dead End Detection:

Identification of failed evolutionary paths:

- **Death Events:** Fitness < 0.5
- **Failure Patterns:** Common failure modes
- **Recovery Strategies:** How to avoid dead ends
- **Prevention:** Early detection methods

8.2 Analysis Tools

Flight Recorder Queries:

```
# Get agent lineage
lineage = flight_recorder.get_lineage(genome_id)

# Analyze fitness trends
trends = flight_recorder.analyze_fitness(generation_range)

# Find convergence points
convergence = flight_recorder.detect_convergence()

# Mutation impact analysis
impact = flight_recorder.analyze_mutation_impact(mutation_type)

# Dead end detection
dead_ends = flight_recorder.detect_dead_ends()
```

Analysis Functions:

Function	Purpose	Output	Performance
<code>get_lineage()</code>	Retrieve agent family tree	Graph structure	O(n)
<code>analyze_fitness()</code>	Fitness trend analysis	Time series data	O(n)
<code>detect_convergence()</code>	Find convergence points	Generation numbers	O(n log n)
<code>analyze_mutation_impact()</code>	Mutation effectiveness	Impact scores	O(n)
<code>detect_dead_ends()</code>	Identify failures	Dead end list	O(n)

Query Performance:

Query Type	Average Time	Complexity	Optimization
Lineage	50ms	O(n)	Indexed by genome_id
Fitness Trends	100ms	O(n)	Cached aggregations
Convergence	200ms	O(n log n)	Statistical sampling
Mutation Impact	150ms	O(n)	Filtered by type
Dead Ends	80ms	O(n)	Pre-computed flags

8.3 Publication Format

WAFT data is suitable for various research publication formats:

Research Paper Formats: - Research papers on artificial cognition - Evolutionary algorithm studies - Agent architecture analysis - Fitness landscape research

Data Export Formats:

Format	Use Case	Tools	Compatibility
JSON	Programmatic analysis	Python, R, JavaScript	Universal
CSV	Spreadsheet analysis	Excel, Google Sheets	Universal
GraphML	Network visualization	Gephi, Cytoscape	Specialized
Newick	Phylogenetic trees	Phylo, TreeView	Biology tools

Publication-Ready Metrics:

Metric	Description	Statistical Test	Significance
Fitness Improvement	Delta over generations	T-test	$p < 0.05$
Mutation Success Rate	Beneficial mutations	Chi-square	$p < 0.01$
Convergence Time	Generations to converge	Regression	$R^2 > 0.8$
Dead End Rate	Failure percentage	Proportion test	Confidence interval

9. Best Practices

9.1 Agent Design

Effective agent design principles for evolutionary success:

Clear Objectives: Define agent goals explicitly

- **Specific Goals:** Clear, measurable objectives
- **Success Criteria:** Defined metrics for success
- **Failure Conditions:** When to terminate
- **Evolution Targets:** What to optimize for

Modular Code: Enable targeted mutations

- **Function Boundaries:** Clear separation of concerns
- **Interface Contracts:** Well-defined APIs
- **Dependency Injection:** Configurable components
- **Mutation Points:** Identified modification sites

Testable: Design for Scint Gym evaluation

- **Error Handling:** Comprehensive error management
- **Validation:** Input/output validation
- **Recovery:** Error recovery mechanisms
- **Observability:** Telemetry hooks

Observable: Include telemetry hooks

- **Event Logging:** Key decision points
- **Metrics Collection:** Performance data
- **State Tracking:** Current state visibility
- **Debug Information:** Troubleshooting data

Agent Design Checklist:

- [] Clear objectives defined
- [] Modular code structure
- [] Error handling implemented
- [] Telemetry hooks added
- [] Testable design
- [] Documentation complete

9.2 Evolution Strategy

Effective strategies for directed evolution:

Diverse Mutations: Explore different code paths

- **Mutation Variety:** Different types of changes
- **Exploration vs Exploitation:** Balance search strategies
- **Random Exploration:** Occasional random mutations
- **Targeted Improvements:** Focused optimizations

Fitness Focus: Prioritize high-fitness variants

- **Selection Pressure:** Strong fitness-based selection
- **Elite Preservation:** Keep best genomes
- **Fitness Thresholds:** Minimum fitness requirements
- **Multi-Objective:** Balance multiple fitness components

Lineage Tracking: Maintain complete records

- **Event Logging:** All evolutionary events
- **Lineage Preservation:** Complete family trees
- **Metadata Capture:** Rich context data
- **Reproducibility:** Ability to replay evolution

Dead End Detection: Identify failed paths early

- **Early Warning:** Detect problems quickly
- **Failure Analysis:** Understand why paths fail
- **Recovery Strategies:** How to avoid dead ends
- **Pattern Recognition:** Learn from failures

Evolution Strategy Comparison:

Strategy	Exploration	Exploitation	Convergence	Use Case
Greedy	Low	High	Fast	Known good paths
Random	High	Low	Slow	Unknown landscape
Balanced	Medium	Medium	Medium	General purpose
Adaptive	Variable	Variable	Optimal	Complex problems

9.3 Scientific Rigor

Maintaining scientific standards in evolutionary experiments:

Complete Records: Log all evolutionary events

- **Event Completeness:** No missing events
- **Context Richness:** Detailed event data
- **Timestamp Precision:** Microsecond accuracy
- **Reproducibility:** Ability to replay

Reproducibility: Use version control and locks

- **Version Control:** Git for code history
- **Dependency Locks:** `uv.lock` for reproducibility
- **Seed Values:** Random seed tracking
- **Environment Documentation:** Complete setup info

Analysis: Regular fitness landscape review

- **Trend Analysis:** Fitness over time
- **Pattern Detection:** Convergence patterns
- **Anomaly Detection:** Unexpected behaviors

- **Statistical Validation:** Significance testing

Documentation: Maintain research notes

- **Experiment Logs:** Detailed experiment records
- **Hypothesis Tracking:** Tested hypotheses
- **Results Documentation:** Findings and conclusions
- **Methodology Notes:** Process documentation

Scientific Rigor Checklist:

- ☐ All events logged
- ☐ Reproducible setup
- ☐ Statistical analysis
- ☐ Documentation complete
- ☐ Peer review ready
- ☐ Data available for sharing

10. Troubleshooting

10.1 Common Issues

Agent Not Evolving:

Symptoms: Agent fitness not improving, no evolution events

Diagnosis Steps: 1. Check fitness scores (must be > 0.5) - Run `waft eval --agent <name>` - Review fitness components - Verify threshold settings 2. Verify Scint Gym evaluation - Check gym logs in `_pyrite/gym_logs/` - Verify quest execution - Check Scint detection 3. Review mutation types - Verify mutations are being applied - Check mutation effectiveness - Review mutation history

Common Causes:

Cause	Symptom	Solution
Low Fitness	Fitness < 0.5	Improve agent code
No Mutations	No spawn events	Check mutation config
Gym Failure	Evaluation errors	Fix gym setup
Resource Limits	Scint < threshold	Accumulate more Scint

Flight Recorder Errors:

Symptoms: Events not logging, lineage broken, data corruption

Diagnosis Steps: 1. Check `_pyrite` directory permissions - Verify write access - Check disk space - Review file permissions 2. Verify event format - Check JSON validity - Verify schema compliance - Review event structure 3. Review lineage structure - Check parent-child links - Verify genome IDs - Review generation numbers

Common Causes:

Cause	Symptom	Solution
Permission Denied	Write failures	Fix permissions
Disk Full	Write errors	Free disk space
Schema Mismatch	Validation errors	Update event format
Corrupted Data	Parse errors	Restore from backup

Dependency Issues:

Symptoms: Import errors, missing packages, version conflicts

Diagnosis Steps: 1. Run `waft sync` - Resolve dependencies - Update lock file - Install packages 2. Check `uv.lock` status - Verify lock file exists - Check for conflicts - Review dependency tree 3. Verify Python version (3.10+) - Check `python --version` - Verify compatibility - Update if needed

Common Causes:

Cause	Symptom	Solution
Missing Lock	Import errors	Run <code>waft sync</code>
Version Conflict	Dependency errors	Resolve conflicts
Python Version	Syntax errors	Upgrade Python
Corrupted Lock	Invalid lock file	Regenerate lock

10.2 Getting Help

Documentation Resources:

- **System Overview:** `docs/SYSTEM_OVERVIEW.md`
- **Architecture:** `docs/research/evolutionary_architecture.md`
- **API Reference:** `docs/` directory
- **Examples:** `examples/` directory

Community Support:

- **GitHub Issues:** Report bugs and request features
- **Discussions:** Community Q&A
- **Documentation:** Comprehensive guides
- **Examples:** Working code samples

Debugging Tools:

Tool	Purpose	Command
<code>waft verify</code>	Check project health	<code>waft verify</code>
<code>waft info</code>	Project information	<code>waft info</code>
<code>waft assess</code>	Epistemic state	<code>waft assess</code>
Flight Recorder	Event inspection	Manual JSONL review

11. Future Roadmap

11.1 Planned Features

Full Evolutionary Cycle Automation: Complete spawn → gym → select automation

- **Automated Spawning:** Intelligent variant generation
- **Batch Evaluation:** Parallel gym testing
- **Auto-Selection:** Automatic fittest selection
- **Continuous Evolution:** Long-running evolution loops

Advanced Mutation Strategies: Genetic algorithm integration

- **Crossover Operations:** Combine parent genomes
- **Mutation Operators:** Specialized mutation types
- **Selection Algorithms:** Tournament, roulette wheel
- **Population Management:** Generation size optimization

Visualization Tools: Phylogenetic tree visualization

- **Interactive Trees:** Browser-based visualization
- **Fitness Landscapes:** 3D fitness visualization
- **Mutation Impact:** Visual mutation analysis
- **Convergence Plots:** Trend visualization

Research Dashboard: Scientific analysis interface

- **Real-time Metrics:** Live evolution monitoring
- **Data Analysis:** Statistical tools
- **Export Functions:** Publication-ready exports
- **Collaboration:** Shared research spaces

11.2 Research Directions

God-Head Emergence: Observing advanced agent intelligence

- **Long-term Evolution:** 1000+ generation experiments
- **Intelligence Metrics:** Cognitive capability measurement
- **Emergence Detection:** Identifying intelligence emergence
- **Capability Analysis:** Understanding advanced behaviors

Fitness Landscape Mapping: Understanding evolution paths

- **Landscape Visualization:** 3D fitness surfaces
- **Path Analysis:** Evolution trajectory mapping
- **Local Optima:** Identifying fitness peaks
- **Escape Strategies:** Breaking out of local optima

Mutation Impact Analysis: Measuring code change effects

- **Causal Analysis:** Mutation → fitness causality
- **Impact Prediction:** Forecasting mutation effects
- **Optimal Mutations:** Identifying best mutation types
- **Mutation Combinations:** Multi-mutation effects

Convergence Studies: Identifying emergent patterns

- **Convergence Detection:** Automatic convergence identification
- **Pattern Recognition:** Emergent behavior patterns
- **Stability Analysis:** Convergence stability
- **Diversity Maintenance:** Preventing premature convergence

12. Case Studies

12.1 Case Study: RefactorAgent Evolution

Scenario: Evolving an agent to improve code refactoring capabilities

Initial State: - Fitness: 0.65 - Generation: 0 (Genesis) - Capabilities: Basic refactoring

Evolution Process:

Generation	Fittest Fitness	Key Mutation	Improvement
0	0.65	-	Baseline
1	0.72	Prompt evolution	Better context understanding
2	0.78	Code optimization	Faster execution
3	0.82	Error handling	Better stability
4	0.85	Architecture refactor	Improved structure
5	0.87	Multi-mutation	Combined improvements

Results: - **Fitness Improvement:** +0.22 (34% increase) - **Generations:** 5 - **Total Variants:** 25 (5 per generation) - **Evolution Time:** 10 minutes - **Key Learnings:** Prompt evolution + code optimization = best results

12.2 Case Study: Multi-Agent Coordination

Scenario: Evolving agents to work together effectively

Setup: 3 agents with complementary capabilities

Evolution Focus: Coordination and communication

Results: - **Fitness Improvement:** +0.15 per agent - **Coordination Score:** 0.82 (high) - **Communication Efficiency:** 85% success rate - **Key Mutation:** Shared context protocol

12.3 Case Study: Long-Term Evolution

Scenario: 100-generation evolution experiment

Duration: 2 weeks continuous evolution

Results: - **Final Fitness:** 0.94 (elite) - **Convergence:** Generation 67 - **Dead Ends:** 12 (12% failure rate) - **Key Patterns:** Architecture improvements most effective

13. Technical Specifications

13.1 Genome ID Algorithm

Algorithm: SHA-256 hash of genome components

Input Components: 1. Source code (Python files) 2. Configuration (JSON) 3. Prompt template (text) 4. Dependencies (list)

Process:

1. Serialize all components to strings
2. Concatenate in deterministic order
3. Apply SHA-256 hashing
4. Return hex digest (64 characters)

Collision Probability: < 0.01% (SHA-256 uniqueness)

13.2 Fitness Calculation

Formula: Weighted sum of components

$$\text{Fitness} = (\text{Stability} \times 0.4) + (\text{Efficiency} \times 0.3) + (\text{Safety} \times 0.3)$$
Component Calculations:

- **Stability:** `scints_stabilized / total_scints`
- **Efficiency:** `successful_calls / total_calls`
- **Safety:** `safety_checks_passed / total_checks`

Precision: Float (0.0 - 1.0), 3 decimal places

13.3 Event Schema

Flight Recorder Event Structure:

```
{
  "event_id": "string (UUID)",
  "timestamp": "ISO 8601",
  "event_type": "SPAWN | MUTATE | GYM_EVAL | DEATH | SURVIVAL",
  "genome_id": "string (SHA-256)",
  "parent_id": "string (SHA-256) | null",
  "generation": "integer (0+)",
  "agent_id": "string",
  "lineage_path": ["array of genome_ids"],
  "payload": {
    "mutation_type": "string",
    "mutation_details": "object",
    "code_diff": "string",
    "config_changes": "object"
  },
  "fitness_metrics": {
    "stability": "float",
    "efficiency": "float",
    "safety": "float",
    "overall": "float"
  },
  "scint_energy": "integer",
  "karma": "float (-1.0 to 1.0)"
}
```

13.4 Performance Specifications

System Performance:

Operation	Target	Typical	Maximum
Spawn Variant	<3s	1-2s	5s
Gym Evaluation	<60s	30-45s	5min
Evolution	<5s	1-3s	10s
Lineage Query	<100ms	50ms	200ms
Event Logging	<10ms	5ms	20ms

Scalability:

Metric	Current	Target	Limit
Agents	100	1000	10000
Generations	100	1000	Unlimited
Events	10K	100K	1M+
Storage	100MB	1GB	10GB+

14. Conclusion

WAFT provides a comprehensive framework for directed evolution of self-modifying AI agents. Through its three pillars—the Substrate (code as DNA), the Physics (Scint System), and the Flight Recorder (telemetry)—WAFT enables scientific research into the physics of artificial cognition.

Key Takeaways:

- 1. **Code is DNA:** Agents evolve through genetic modifications
- 2. **Fitness Drives Evolution:** Scint System provides natural selection
- 3. **Complete Observability:** Flight Recorder enables scientific analysis
- 4. **Directed Evolution:** Guided by fitness, not random mutation

The Promise: "Don't just build agents. Breed them."

Future Vision: WAFT continues to evolve, with plans for advanced mutation strategies, visualization tools, and long-term evolution experiments. The framework is designed to scale from small experiments to large-scale evolutionary research, enabling the scientific community to study artificial cognition through rigorous, reproducible experiments.

References

- 1. WAFT Repository: <https://github.com/ctavolazzi/waft>
- 2. System Overview: docs/SYSTEM_OVERVIEW.md
- 3. Unified Genesis Protocol: docs/UNIFIED_GENESIS_PROTOCOL.md
- 4. AI SDK Vision: docs/AI_SDK_VISION.md
- 5. Evolutionary Architecture: docs/research/evolutionary_architecture.md
- 6. State of the Art: docs/research/state_of_art_2026.md

- 7. Decision Engine Architecture: `docs/DECISION_ENGINE_ARCHITECTURE.md`
 - 8. Being System Documentation: `src/waft/being.py`
 - 9. Flight Recorder Schema: `src/waft/core/agent/state.py`
 - 10. Scint System Implementation: `core/hub/`
-

License

MIT

Version: 1.0.0

Last Updated: 2026-01-12

Status: Comprehensive Edition with Charts and Detailed Specifications